



Political Patricians (patricians)

Un censor romano está monitoreando N patricios con lealtades políticas cuestionables. Cada patricio pertenece a **exactamente una** facción política secreta, y cada facción tiene uno o más patricios. El censor quiere determinar qué patricios pertenecen a cada facción.

Para conseguirlo, el censor va a organizar sucesivos banquetes en que los N patricios están invitados. En cada banquete, los patricios se pueden sentar en distintas mesas, y el censor controla la disposición de asientos. Un patricio se siente *solo* si no hay otros miembros de la misma facción política sentados en su mesa.

Como maestro de ceremonias, el censor va a saber después de cada banquete qué patricios se sintieron solos. Tu tarea es ayudar al censor a determinar las lealtades políticas de los patricios sin organizar demasiados banquetes.

Implementación

Vas a tener que implementar un único fichero `.cpp` fuente.



Entre los ficheros adjuntos a esta tarea, vas a encontrar una plantilla `patricians.cpp` con un ejemplo de implementación.

Vas a tener que implementar la siguiente función:

C++	<code>vector<int> find_factions(int N)</code>
-----	---

- El entero N representa el número de patricios. Los patricios están numerados de 0 a $N - 1$.
- La función debe devolver un vector P de N enteros con las facciones políticas de cada patricio. $P[i]$ tiene que ser igual a $P[j]$ si y sólo si los patricios i y j pertenecen a la misma facción política. No hay más restricciones sobre los valores de los enteros de P .

Tu programa puede llamar a la siguiente función, que ya está definida en el juez:

C++	<code>vector<bool> organize_banquet(const vector<int>& Q)</code>
-----	--

- El vector Q de N enteros representa una disposición de asientos: el patricio i se va a sentar en la mesa $Q[i]$. No hay más restricciones sobre los valores de los enteros de Q : las mesas se pueden numerar de cualquier forma, y la única información relevante es que si $Q[i] = Q[j]$, entonces el patricio i y el patricio j se van a sentar en la misma mesa.
- La función va a retornar un vector A de N booleanos. $A[i]$ va a ser cierto si y solo si el patricio i se siente solo en el banquete.

El juez **no es adaptativo** y cada envío es juzgado en el mismo conjunto de casos de prueba. Es decir, las facciones políticas de cada patricio están fijas de antemano en cada caso de prueba y no depende de las llamadas a `organize_banquet` hechas por tu programa.

Sample Grader

Entre los ficheros adjuntos a esta tarea vas a encontrar una versión simplificada del juez usada durante la evaluación, que puedes usar para comprobar tus soluciones localmente. El juez de muestra lee de `stdin`, llama la función `find_factions` y escribe en `stdout` usando el siguiente formato.

El fichero de entrada consta de 2 líneas, conteniendo:

- Línea 1: el entero N .
- Línea 2: N enteros describiendo la facción política de cada patricio.

La salida consta de una sola línea, conteniendo la puntuación obtenida por tu solución en el caso de prueba. Si la puntuación es 0, imprime un mensaje breve al `stderr` con la razón (distribución de facciones incorrecta, consulta inválida, o demasiadas consultas).

Restricciones

- $4 \leq N \leq 2000$.

Puedes llamar a `organize_banquet` un máximo de 30000 veces.

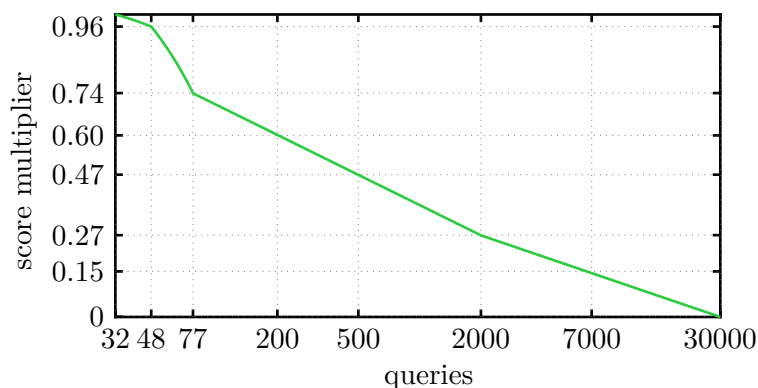
Puntuación

Tu programa va a ser comprobado contra varios casos de prueba agrupados en subtareas. El multiplicador de puntuación asociado de una subtarea va a ser el mínimo multiplicador de puntuación obtenido en cada uno de los casos de prueba.

- **Subtask 0 [0 puntos]**: Caso de prueba de muestra.
- **Subtask 1 [19 puntos]**: Hay exactamente dos facciones políticas.
- **Subtask 2 [29 puntos]**: Cada facción política consta de exactamente dos patricios.
- **Subtask 3 [52 puntos]**: Sin restricciones adicionales.

El multiplicador de puntuación obtenido en cada caso de prueba se determina de la forma siguiente.

Number of queries	Score multiplier
≤ 32	1.0
$33 \leq x < 48$	$1.08 - 0.0025x$
48	0.96
$48 < x < 77$	$1.32413 - 0.00758x$
77	0.74
$77 < x < 2000$	$1.366 - 0.1 \log_2(x)$
2000	0.27
$2000 < x \leq 30000$	$1.027 - 0.0691 \log_2(x)$



La puntuación obtenida en cada caso de prueba es el multiplicador de puntuación mínimo obtenido sobre todos los casos de prueba, multiplicado por la puntuación base de la subtarea.

Ejemplos de entrada/salida

Grader	Solución
<code>find_factions(5)</code>	
	<code>organize_banquet([0,0,1,1,2])</code>
<code>return [1,1,1,1,1]</code>	
	<code>organize_banquet([0,0,0,1,1])</code>
<code>return [0,1,0,1,1]</code>	
	<code>organize_banquet([0,1,2,1,1])</code>
<code>return [1,0,1,0,1]</code>	
	<code>return [0,1,0,1,2]</code>

Grader	Solución
<code>find_factions(4)</code>	
	<code>organize_banquet([1,1,0,0])</code>
<code>return [1,1,1,1]</code>	
	<code>organize_banquet([1,1,1,2])</code>
<code>return [0,1,0,1]</code>	
	<code>organize_banquet([0,1,1,1])</code>
<code>return [1,0,1,0]</code>	
	<code>return [0,1,0,1]</code>

Explicación

En el primer ejemplo, los patricios 0 y 2 pertenecen a la misma facción política, los patricios 1 y 3 pertenecen a otra facción distinta, y el patricio 4 está él solo en su propia facción. En el primer banquete, todos los patricios se sienten solos. En el segundo banquete, los patricios 0 y 2 no están solos porque están sentados juntos. En el tercer banquete, los patricios 1 y 3 no están solos porque están sentados juntos.